

MOBILE APPLICATION DEVELOPMENT

SYLLABUS

Credits: 4

Semester: V

Subject Code: CS21503

Objectives

- To understand the fundamentals of Android Operating Systems.
- To demonstrate the Android software development tools.
- To develop software with reasonable complexity on mobile platform.
- To deploy software to mobile devices.
- To debug programs running on mobile devices.

Outcome:

CO1: Compare different mobile application models/architectures and patterns

CO2: Apply a mobile development framework to the development of a mobile application.

CO3: Explain components and structure of a mobile development framework.

CO4: Develop advanced Java programming competency.

CO5: Develop final application.

UNIT 1

12 hours

Overview of Android

- Introduction to Android – History of Android Platform 1
- Introduction to IOS, Android APIs, Android Architecture 2
- Framework, Features of Android, Android Components, Manifest File. 2
- Downloading and Installing Android – Android SDK, Android Virtual Device, 3
- Android Physical Device, Development Environment. 3
- Developing and executing the First Android Application – 2
- Using Eclipse IDE to create an application, 2
- Running the Application, Exploring the Application, Using command-line tools. 2

UNIT 2

12 hours

- Working with Activities – Creating an Activity, Starting an Activity 2
- Lifecycle of an activity 2
- Applying themes and styles to the activity, Displaying dialogs in the activity 2
- Hiding the title of the activity.
- Intents – Exploring Intent objects, Intent resolution, Intent Filters, Linking the 2
- activities using Intent, Obtaining results from an Intent
- Passing data Using an Intent Object. 1

- Fragments – Life cycle of a fragment, Implementation, using the Intent Object to Invoke Built-in-Application. 3

UNIT 3

12 hours

- User Interface using Views and View Groups 1
- Working with view groups – Linear layout, Relative layout, Scroll layout, Table layout, Frame layout, Tab layout. 3
- Working with views – Text , Edit text, Button, Radio button, Checkbox, Image Button, Toggle button, Rating bar. 3
- Binding Data with the Adapter View Class – List view, Spinner, Gallery view. Implementing Screen Orientation . 3
- Handling UI Events – With Activities, With Views. 2

UNIT 4

12 hours

- Handling Pictures and Menus with Views 2
- Working with Image views – Displaying image in gallery and grid. 2
- Designing context menu, Analog and Digital clock views. 2
- Embedding web browser in the Activity. 2
- Notifications – Creating and Displaying notifications. 2
- DisplayingToasts. 2

UNIT 5

12 hours

- Persistent Storage -Using application specific folders and files, Creating files, 2
- Reading data from files, listing contents of a directory shared preferences 2
- Creating shared preferences 2
- Database-Introduction to SQLite database, creating and opening a database, 2
- creating and opening a database, creating tables 2
- Inserting retrieving and deleting data, Registering Content Providers. 2
- Using content providers – insert, retrieve and update. 2

ESSENTIAL READINGS:

1. Pradeep Kothari. Android Application Development- Black Book. DreamTech press.2017
2. Jerome (J.F) DiMarzio. Android – A programmer’s guide – Mc Graw Hill.2008.
3. Dawn riffiths & David Griffiths. Head First Android Development-O’reilly-second edition.

UNIT-1

OVERVIEW OF ANDROID

INTRODUCTION TO ANDROID

What is Android



Android is a software package and linux based operating system for mobile devices such as tablet computers and smartphones.

It is developed by Google and later the OHA (Open Handset Alliance). Java language is mainly used to write the android code even though other languages can be used.

The goal of android project is to create a successful real-world product that improves the mobile experience for end users.

There are many code names of android such as Lollipop, Kitkat, Jelly Bean, Ice cream Sandwich, Froyo, Ecliar, Donut etc

What is Open Handset Alliance (OHA)

It's a consortium of 84 companies such as google, samsung, AKM, synaptics, KDDI, Garmin, Teleca, Ebay, Intel etc.

It was established on 5th November, 2007, led by Google. It is committed to advance open standards, provide services and deploy handsets using the Android Platform.

Features of Android

After learning what is android, let's see the features of android. The important features of android are given below:

- 1) It is open-source.
- 2) Anyone can customize the Android Platform.
- 3) There are a lot of mobile applications that can be chosen by the consumer.
- 4) It provides many interesting features like weather details, opening screen, live RSS (Really Simple Syndication) feeds etc.

It provides support for messaging services(SMS and MMS), web browser, storage (SQLite), connectivity (GSM, CDMA, Blue Tooth, Wi-Fi etc.), media, handset layout etc.

Categories of Android applications

There are many android applications in the market. The top categories are:

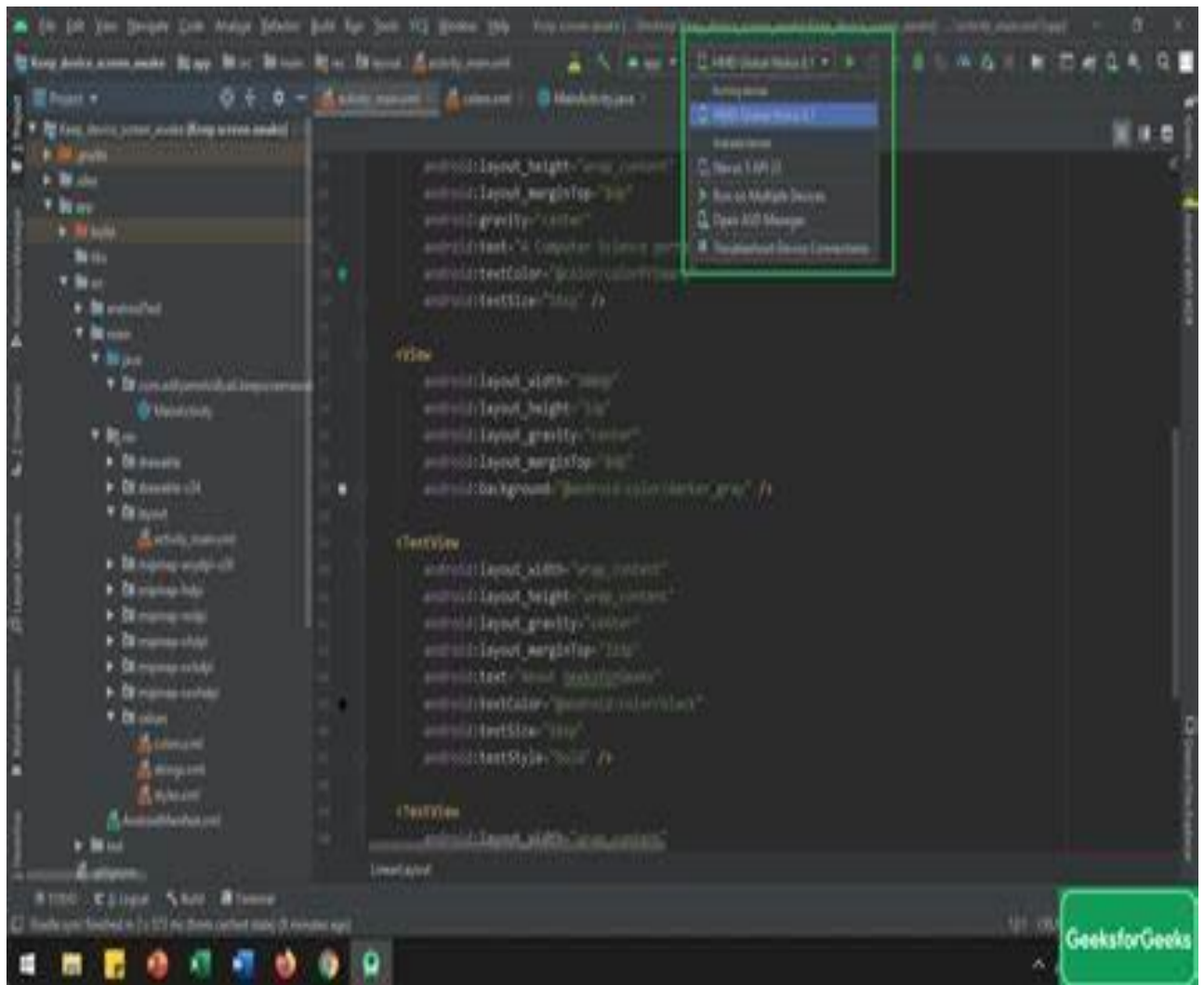
-
- Entertainment
- Tools
- Communication
- Productivity
- Personalization
- Music and Audio
- Social
- Media and Video
- Travel and Local etc.

History of Android

The history and versions of android are interesting to know. The code names of android ranges from A to J currently, such as **Aestro, Blender, Cupcake, Donut, Eclair, Froyo, Gingerbread, Honeycomb, Ice CreamSandwitch, JellyBean, KitKat** and **Lollipop**.

Let's understand the android history in a sequence.

- 1) Initially, **Andy Rubin** founded Android Incorporation in Palo Alto, California, United States in October, 2003.
- 2) In 17th August 2005, Google acquired android Incorporation. Since then, it is in the subsidiary of Google Incorporation.
- 3) The key employees of Android Incorporation are **Andy Rubin, Rich Miner, Chris White** and **Nick Sears**.
- 4) Originally intended for camera but shifted to smart phones later because of low market for camera only.
- 5) Android is the nick name of Andy Rubin given by coworkers because of his love to robots.
- 6) In 2007, Google announces the development of android OS.
- 7) In 2008, HTC launched the first android mobile.



The word 'Android' itself means 'Human-like robot'. This name is very suitable for the brand, as well as the logo designed according to the concept of a robot assistant, which helps you use your phone (even though the logo itself isn't terribly human-like

Android Architecture

Android architecture contains a different number of components to support any Android device's needs. Android software contains an open-source Linux Kernel having a collection of a number of C/C++ libraries which are exposed through application framework services.

Components of Android Architecture

The main components of Android architecture are the following:-

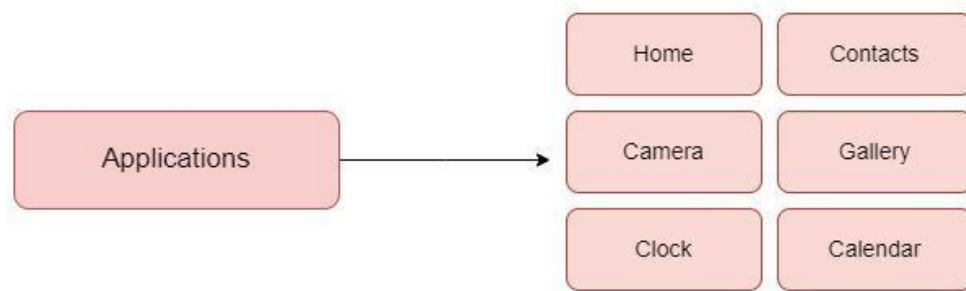
- Applications
- Application Framework
- Android Runtime
- Platform Libraries
- Linux Kernel

Pictorial representation of Android architecture with several main components and their sub-components



1. Applications

Applications is the top layer of android architecture. The pre-installed applications like home, contacts, camera, gallery etc and third party applications downloaded from the play store like chat applications, games etc. will be installed on this layer only. It runs within the Android run time with the help of the classes and services provided by the application framework.

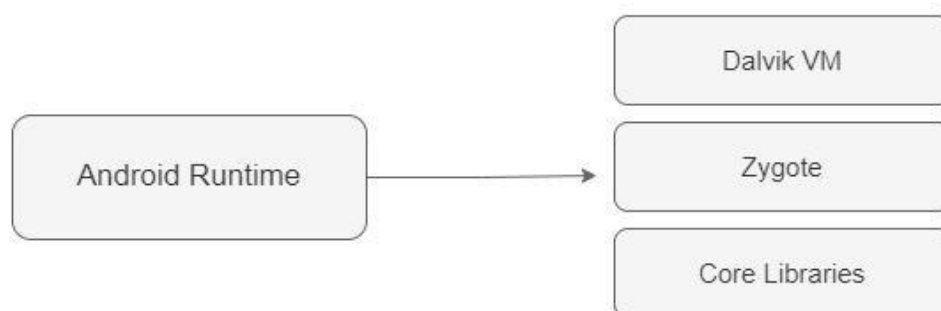


2. Application framework

Application Framework provides several important classes which are used to create an Android application. It provides a generic abstraction for hardware access and also helps in managing the user interface with application resources. It includes different types of services activity manager, notification manager, view system, package manager etc. which are helpful for the development of our application according to the prerequisite.

4. Application runtime

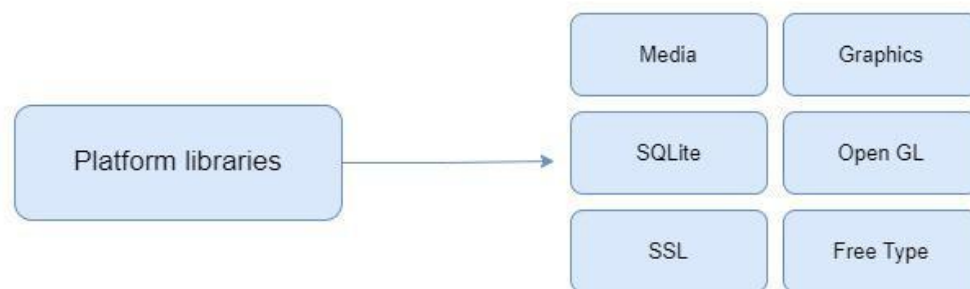
Android Runtime environment is one of the most important part of Android. It contains components like core libraries and the Dalvik virtual machine(DVM). Mainly, it provides the base for the application framework and powers our application with the help of the core libraries. It depends on the layer Linux kernel for threading and low-level memory management. The core libraries enable us to implement android applications using the standard JAVA or Kotlin programming languages.



5. Platform libraries

The Platform Libraries includes various C/C++ core libraries and Java based libraries such as Media, Graphics, Surface Manager, OpenGL etc. to provide a support for android development.

- **Media** library provides support to play and record an audio and video formats.
- **Surface manager** responsible for managing access to the display subsystem.
- **SGL** and **OpenGL** both cross-language, cross-platform application program interface (API) are used for 2D and 3D computer graphics.
- **SQLite** provides database support and **FreeType** provides font support.
- **Web-Kit** This open source web browser engine provides all the functionality to display web content and to simplify page loading.
- **SSL (Secure Sockets Layer)** is security technology to establish an encrypted link between a web server and a web browser.

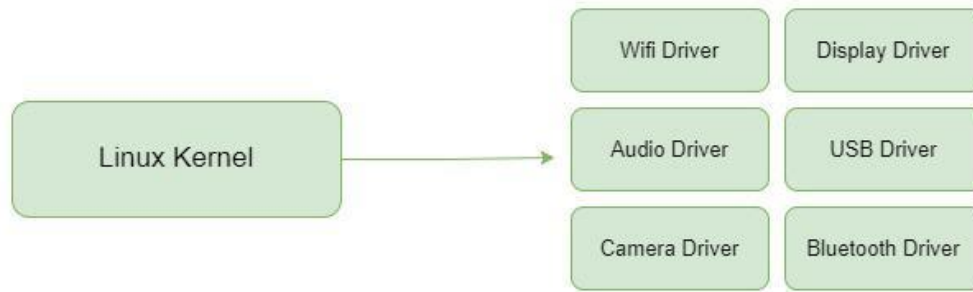


6. Linux Kernel

Linux Kernel is heart of the android architecture. It manages all the available drivers such as display drivers, camera drivers, Bluetooth drivers, audio drivers, memory drivers, etc. which are required during the runtime. It is responsible for management of memory, power, devices etc. The features of Linux kernel are:

- **Security:** The Linux kernel handles the security between the application and the system.
- **Memory Management:** It efficiently handles the memory management thereby providing the freedom to develop our apps.
- **Process Management:** It manages the process well, allocates resources to processes whenever they need them.
- **Network Stack:** It effectively handles the network communication.

- **Driver Model:** It ensures that the application works properly on the device and hardware manufacturers responsible for building their drivers into the Linux build.



Android Version List: Android 1.0 – Android Version 15

In this Android version list, we have collated all Android version names along with the year they were released. If you wish to learn more about Android, you can check out this online [Android course](#).

Android Version	Android Version Names	Release Year	Features	symbol
Android Versions 1.0 – 1.1	No codename	2008	Google apps like Gmail, YouTube, Calendar, Maps, Search, Instant Messaging, and many more.	

Android Version 1.5	Cupcake	2009	Android introduced the first on-screen keyboard as people moved to touchscreen smartphones from keypad-style handsets.introduce d the platform's first-ever video recording option.	 Cupcake Andriod 1.5
Android Version 1.6	Donut	2009	IT transitioned towards bigger screens and resolution phones. Donut came with built-in support for CDMA networks, which helped Android grow quickly. CDMA stands for Code Division Multiple Access.select multiple photos for deletion at a time.	 Donut Android 1.6
Android Versions 2.0 – 2.1	Eclair	2009	It introduced SMS, MMS, voice-guided turn-by-turn navigation and real-time traffic information, pinch-to-zoom capability (which only Apple had at that time),	

			Bluetooth 2.1, fixed minor API, and a few bug fixes.	
Android Version 2.2	Froyo	2010	it introduced voice actions, which allowed users to perform basic functions like speaking a command, making notes, and getting directions.	
Android Version 2.3	Gingerbread	2010	Gingerbread supported an extra-large screen, a simplified interface, enhanced copy/paste functions.	
Android Versions 3.0 – 3.2	Honeycomb	2011	designed to make the most of the tablet's widescreen space.	
Android Version 4.0	Ice Cream Sandwich	2011	It carried over the card-like appearance from Honeycomb and also introduced swiping, common for navigating across the OS.	

Android Versions 4.1 – 4.3	Jelly Bean	2012	it improved accessibility and offered multiple features like screen lock, bug removal,	
Android Version 4.4	KitKat	2013	“OK Google” support, offline music support, smart caller ID, better application compatibility, and many other built-in features.	
Android Versions 5.0 – 5.1	Lollipop	2014	Lollipop improved the ‘OK Google’ command support. This feature’s voice activation was extended to work even when the device’s screen was off.	
Android Version 6.0	Marshmallow	2015	Marshmallow introduced support for fingerprint readers, USB-C, App Standby feature, Doze mode to save battery life,	

Android Versions 7.0 – 7.1	Nougat	2016	Android Nougat is popular for releasing Google Assistant. They offered split-screen mode, a Data Saver feature, file-based encryption, battery usage alerts, a zoom-in screen, and many more features.	
Android Versions 8.0 – 8.1	Oreo	2017	Oreo brought in some of the best features, like picture-in-picture support, adaptive icons, 2x booting speed, Google Play Protect, a notification snoozing option, and many other features.	
Android Version 9	Pie	2018	it introduced many security and privacy enhancements and intelligent systems to manage power and screen brightness.	
Android Version 10	Android 10	2019	It also brought more silent	NO SYMBOL

			improvements and enhancements like granular permissions, control over location data,	
Android Version 11	Android 11	2020	it simplified all the panels and introduced conversation-style notifications, native screen-recording features, and connected-device controls.	- NO SYMBOL
Android Version 12	Android 12	2021	It also renewed its focus on widgets, other improvements, and easy accessibility with separate AI sections to function independently.	- NO SYMBOL
Android Version 13	Android 13	2022	It is a stable OS launched in 2022 with a new interface design for handheld devices.	- NO SYMBOL
Android Version 14	Android 14	2023	It aims to provide rich camera and media experiences with Ultra HDR, lossless USB audio,	- NO SYMBOL

			and more camera extensions.	
Android Version 15	Android 15	2024	It includes SDK and tools, APIs, system images, and Flash-to-OTA updates for Google Pixel.	NO SYMBOL

INTRODUCTION TO IOS

iOS is an abbreviation for the **iPhone operating system**. iOS is the operating system that runs on many of Apple's mobile devices, including the iPhone and iPod Touch. It is the world's second most popular mobile OS, trailing only Android.

It is the basis for three other Apple operating systems: **iPadOS**, **tvOS**, and **watchOS**. It is a part of proprietary software. Some are open source under the **Apple Public Source License** and other licenses.

The iOS os was first released in **2007** for the first-generation iPhone and has since been updated to support more Apple devices, including the iPod Touch and iPad..Apple releases a new major version of iOS every year. On **September 20, 2021**, the current stable version, **iOS 15**, was released to the public.

Features of the iOS

There are various features of iOS. Some main features of the iOS are as follows:

1. The iPhone OS has multitasking capabilities. It all started with the **iPhone 4** and **iPhone 3GS**. You may quickly switch from one app to another by using the multitasking function on an iOS device or a multi-finger motion on an iPad.
2. Apple's iCloud service is a cloud-based data storage service. It works on all Apple devices, has some Windows compatibility, and manages most

operations in the background. It is quite secure. It includes a backup feature to ensure that the user's data is not lost.

3. Apple offers an online mapping service that may be utilized as the default map system for iOS devices
4. Apple also provides the Core Bluetooth framework, and it offers classes for connecting to Bluetooth-enabled low-energy wireless technology.
5. A gyroscope is a device that measures the rate at which a device rotates around a spatial axis. Many iOS devices include a three-axis gyroscope that delivers rotation data in all three axes.

Android Application components

App components are the essential building blocks of an Android app. Each component is an entry point through which the system or a user can enter your app. Some components depend on others.

There are four types of app components:

- Activities
- Services
- Broadcast receivers
- Content providers

Each type serves a distinct purpose and has a distinct lifecycle that defines how a component is created and destroyed. The following sections describe the four types of app components.

1.Activities

An *activity* is the entry point for interacting with the user. It represents a single screen with a user interface. For example, an email app might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails. Although the activities work together to form a cohesive user experience in the email app, each one is independent of the others.

2.Services

A *service* is a general-purpose entry point for keeping an app running in the background for all kinds of reasons. It is a component that runs in the background to perform long-running operations or to perform work for

3.Broadcast receivers

A *broadcast receiver* is a component that lets the system deliver events to the app outside of a regular user flow so the app can respond to system-wide broadcast announcements. Because broadcast receivers are another well-defined entry into the app, the system can deliver broadcasts even to apps that aren't currently running.

4.Content providers

A *content provider* manages a shared set of app data that you can store in the file system, in a SQLite database, on the web, or on any other persistent storage location that your app can access. Through the content provider, other apps can query or modify the data, if the content provider permits it.

The manifest file

Before the Android system can start an app component, the system must know that the component exists by reading the app's *manifest file*, `AndroidManifest.xml`. The manifest does a number of things in addition to declaring the app's components, such as the following:

- Identifies any user permissions the app requires, such as internet access or read-access to the user's contacts.
- Declares the minimum [API level](#) required by the app, based on which APIs the app uses.
- Declares hardware and software features used or required by the app, such as a camera, Bluetooth services, or a multitouch screen.
- Declares API libraries the app needs to be linked against (other than the Android framework APIs), such as the [Google Maps library](#).

DOWNLOAD AND INSTALL ANDROID

Steps to Install Android Studio on Windows

Step 1: Head over to **this link** to get the Android Studio executable or zip file.

Step 2: Click on the **Download Android Studio** Button.



Android Studio provides the fastest tools for building apps on every type of Android device.

DOWNLOAD ANDROID STUDIO

4.1.3 for Windows 64-bit (896 MiB)

Click on the “I have read and agree with the above terms and conditions” checkbox followed by the download button.

A screenshot of the Android Studio download page. The page has a white background with a grey border. At the top, it says "Download Android Studio". Below that, it says "Before downloading, you must agree to the following terms and conditions." There is a scrollable box containing the "Terms and Conditions" for the Android Software Development Kit License Agreement. The terms include an introduction and definitions for "Android" and "compatible implementation". At the bottom of the scrollable box, there is a checkbox labeled "I have read and agree with the above terms and conditions". Below the checkbox is a button labeled "DOWNLOAD ANDROID STUDIO FOR WINDOWS". At the very bottom, the filename "android-studio-ide-173.4819257-windows.exe" is displayed.

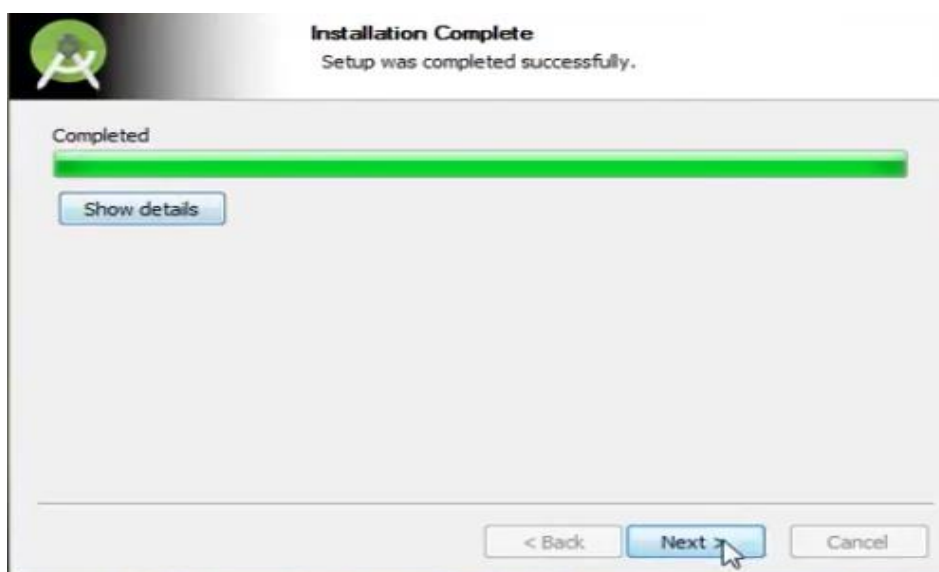
Click on the Save file button in the appeared prompt box and the file will start downloading.

Step 3: After the downloading has finished, open the file from downloads and run it. It will prompt the following dialog box.



Click on next. In the next prompt, it'll ask for a path for installation. Choose a path and hit next.

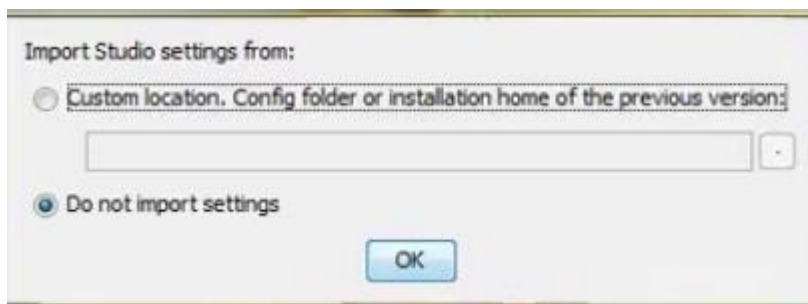
Step 4: It will start the installation, and once it is completed, it will be like the image shown below.



Click on next.



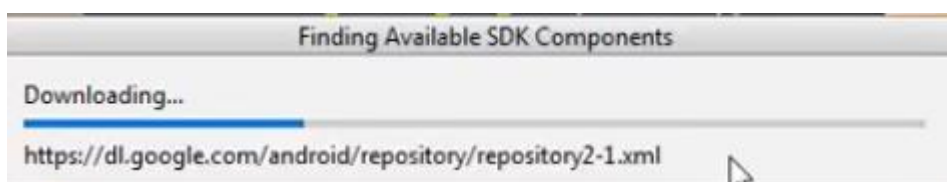
Step 5: Once “**Finish**” is clicked, it will ask whether the previous settings need to be imported [if the android studio had been installed earlier], or not. It is better to choose the ‘Don’t import Settings option’.



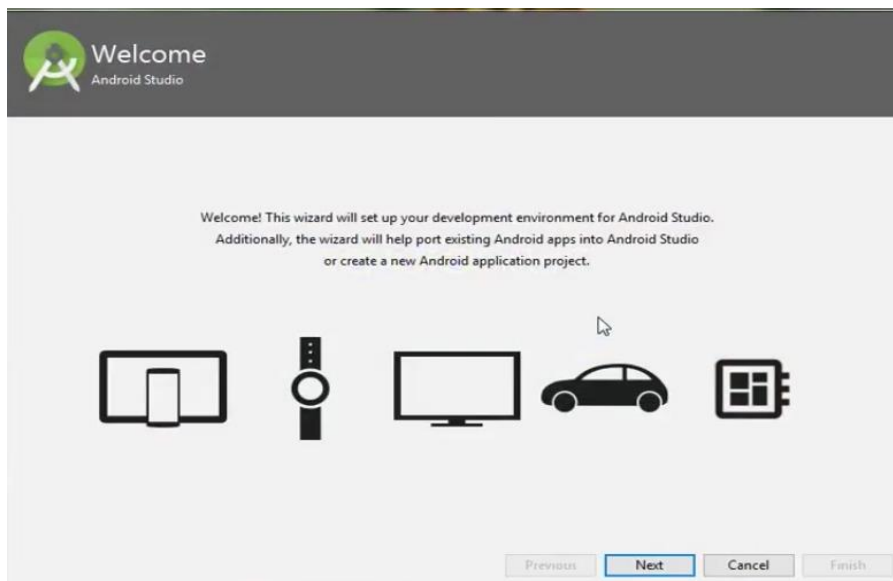
Click the **OK** button.

Step 6: This will start the Android Studio.

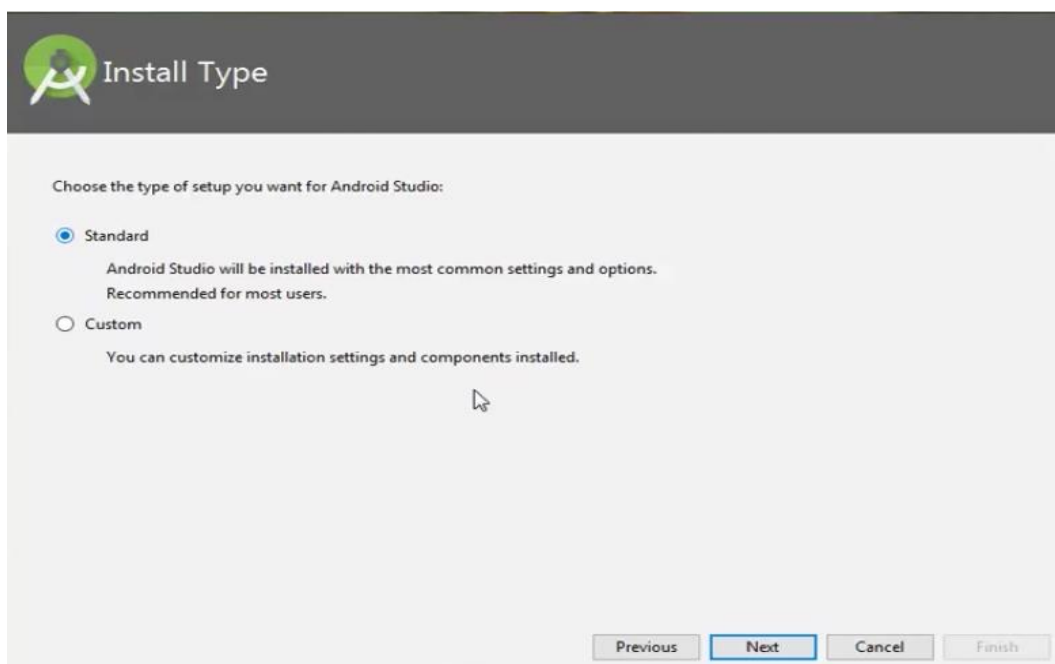
Meanwhile, it will be finding the available SDK components.



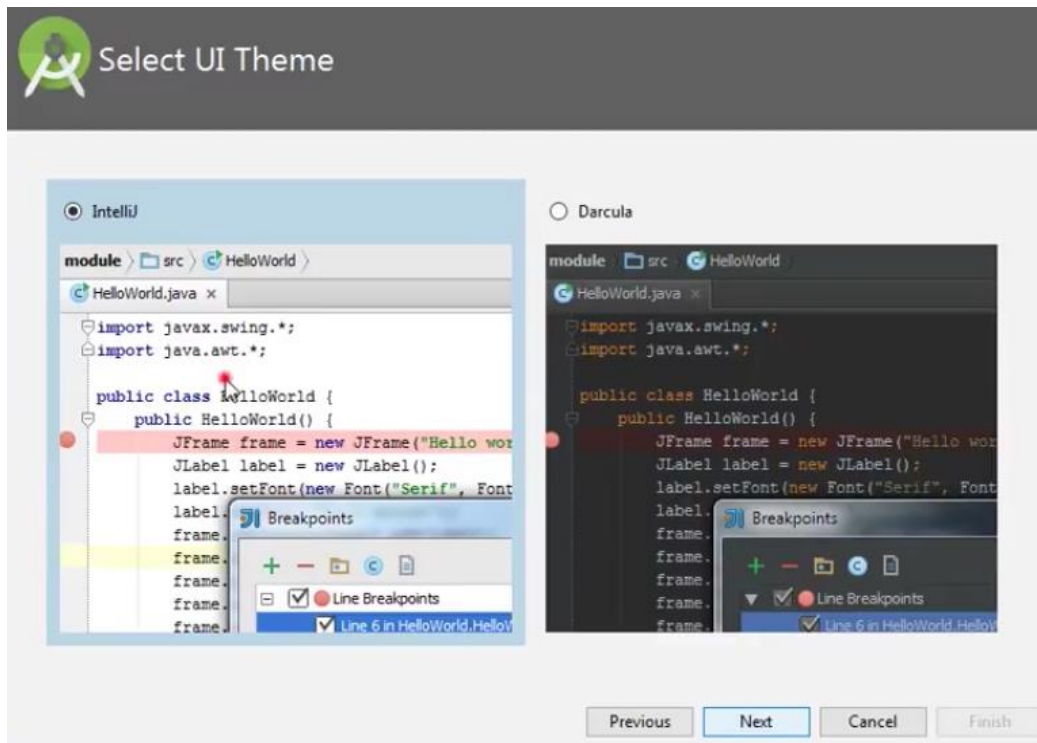
Step 7: After it has found the SDK components, it will redirect to the Welcome dialog box.



Click on **Next**.

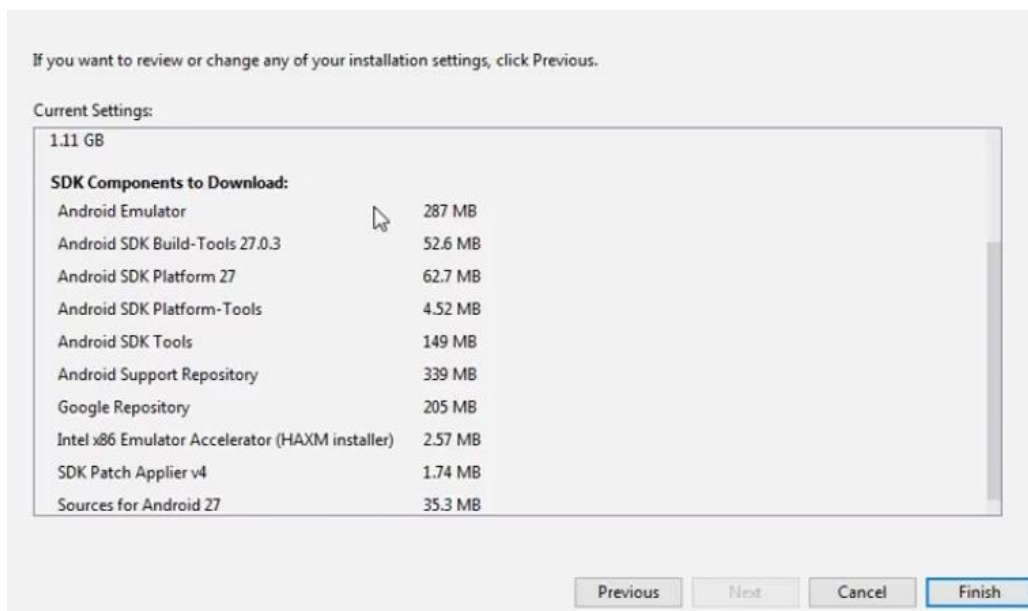


Choose Standard and click on Next. Now choose the theme, whether the **Light** theme or the **Dark** one. The light one is called the **IntelliJ** theme whereas the dark theme is called **Dracula**. Choose as required.

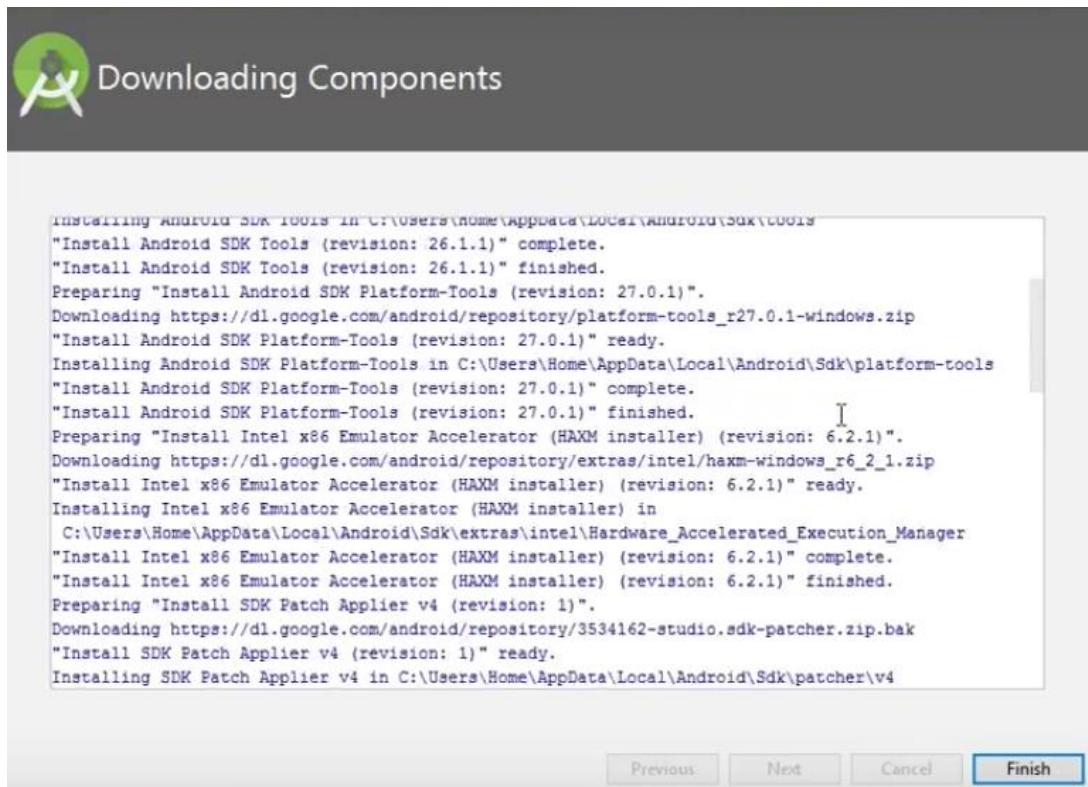


Click on the **Next** button.

Step 8: Now it is time to download the SDK components.



Click on Finish. Components begin to download let it complete.



The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

Step 9: Click on **Start a new Android Studio project** to build a new app.



To run your first android app in Android Studio you may refer to [Running your first Android app](#).

DEVELOPING AND EXECUTING ANDROID APPLICATION

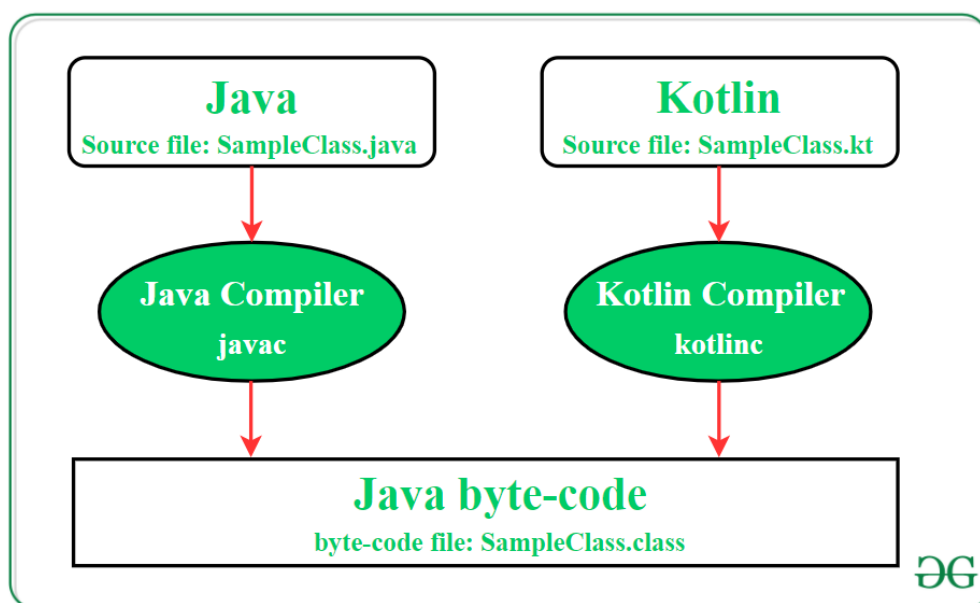
Developing an [android](#) application involves several processes that happen in a sequential manner. After writing the source code files, when developers click the **Run button** on the [Android studio](#), plenty of operations and process starts at the backend.

Step 1: Building the APK File

1. Code Compilation

The android application source files are written in either [Java\(*.java files\)](#) or [Kotlin\(*.kt files\)](#) programming languages. Syntax of writing the code in these 2 languages are different but their compilation process is almost the same. Both programming languages generate code that can be compiled to **Java byte-code** which is executable on [JVM\(Java Virtual Machine\)](#).

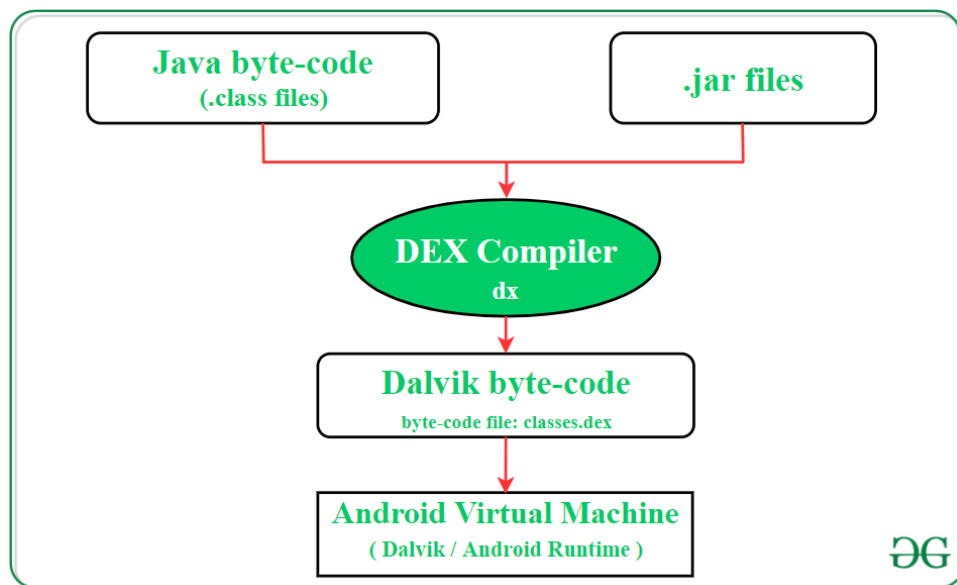
In an android environment, the process begins with the compilation of Java/Kotlin source code into the [Java class file](#). The class files have the extension ***.class** and it contains the java byte-code(represents Java assembly). This compilation task is carried out by the **javac** and **kotlinc** compilers for the Java and Kotlin language code respectively.



2. Conversion into Dalvik bytecodes

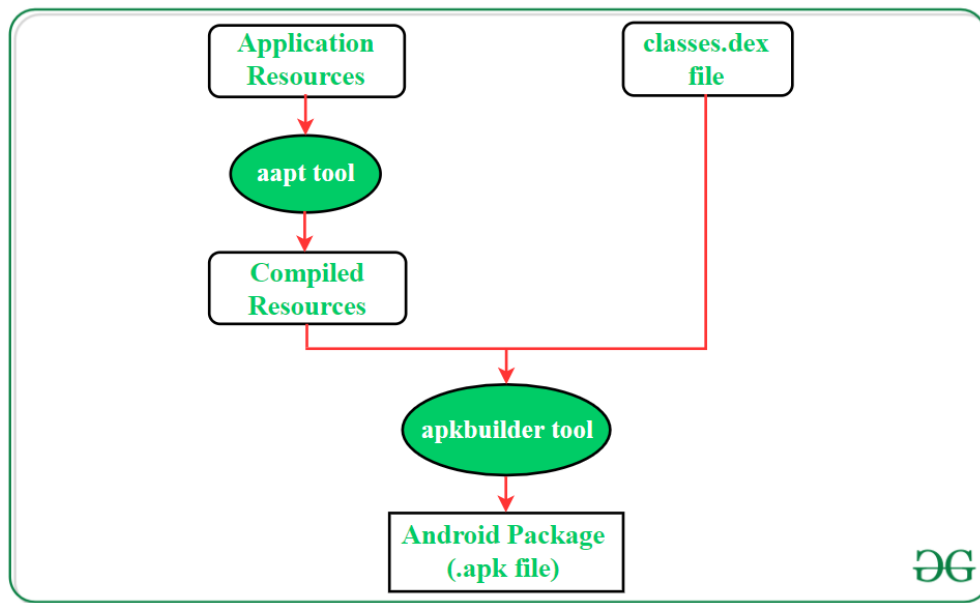
The generated Java class(*.class) file in the previous step is executable on Java Virtual Machine(JVM) as it contains the standard **Oracle JVM Java byte-codes**. However, this code format is not suitable for Android devices and thus Android has its own unique byte-code format known as **Dalvik byte-code**.

Dex compiler translates the Java byte-code into the Dalvik byte-code that are machine-code instructions for a theoretical processor. This file is now executable on the virtual machine in the Android operating system known as **Android Runtime**(or [Dalvik Virtual Machine\(DVM\)](#))



3. Generating .apk file

[Resource files](#) of the android application like images, fonts, XML layouts, etc. are transformed into a single compiled resource unit by the **Android Asset Packaging Tool(aapt)**. Further, the compiled resource unit along with the **classes.dex** file is compressed by the **apkbuilder tool** and a zip-like file is created that is termed as **Android Package(.apk file)**. The generated .apk file contains all necessary data to run the Android application.



4. App Distribution

The .apk file generated in the previous step is a ready-to-use application package and developers can use this file for the purpose of app distribution. However, to distribute and [publish the application through Google Play Store](#), developers need to sign it

Step 2: Deploy the Application

1. Establish the ADB Server

Android Debug Bridge(ADB) deploys an application to Android devices. It is a command-line tool that acts as an interface and facilitates developers to communicate with an android device. To start the deployment, the **ADB client** will first check whether the **ADB server process** is already running on the device.

2. Transfer .apk file to the Device

The ADB command transfers the .apk file into the local file system of the target Android device. The app location in the device file system is defined by its package name. For example, if the application package is *com.example.sampleapp*, then its .apk file will be located in the path */data/app/com.example.sampleapp*.

Step 3: Run the Application

1. App launch request

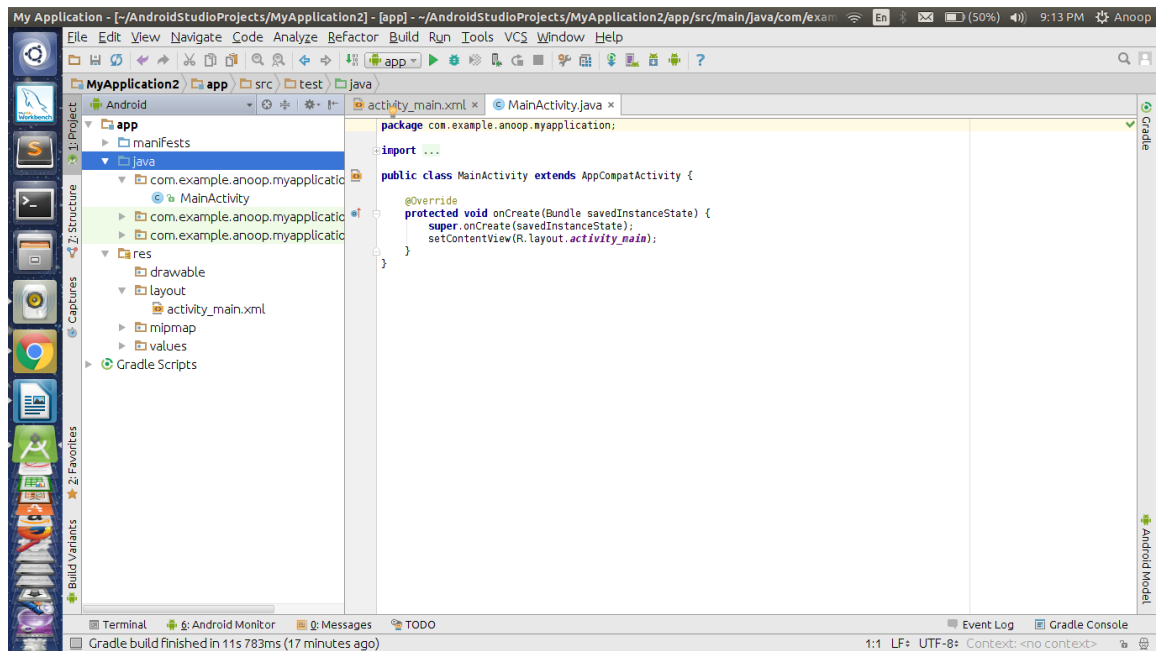
Zygote process is the parent to all Android apps and it launches an application when a user makes the request to do so. The zygote is a special kind of Android OS process which enables code sharing between different instances that run across Android virtual devices(Dalvik/Android Runtime). Those resources, classes, and code libraries which possibly required by any application at runtime are preloaded in the memory space of the zygote process.

2. Conversion of the .dex code to native OAT format

When a new application is installed, the Android optimize the app data and generates a corresponding OAT file. This file is created by the Android OS to accelerate the application loading time. The process to generate the OAT file starts with the extraction of **classes.dex** file present inside the **.apk** file of the application.

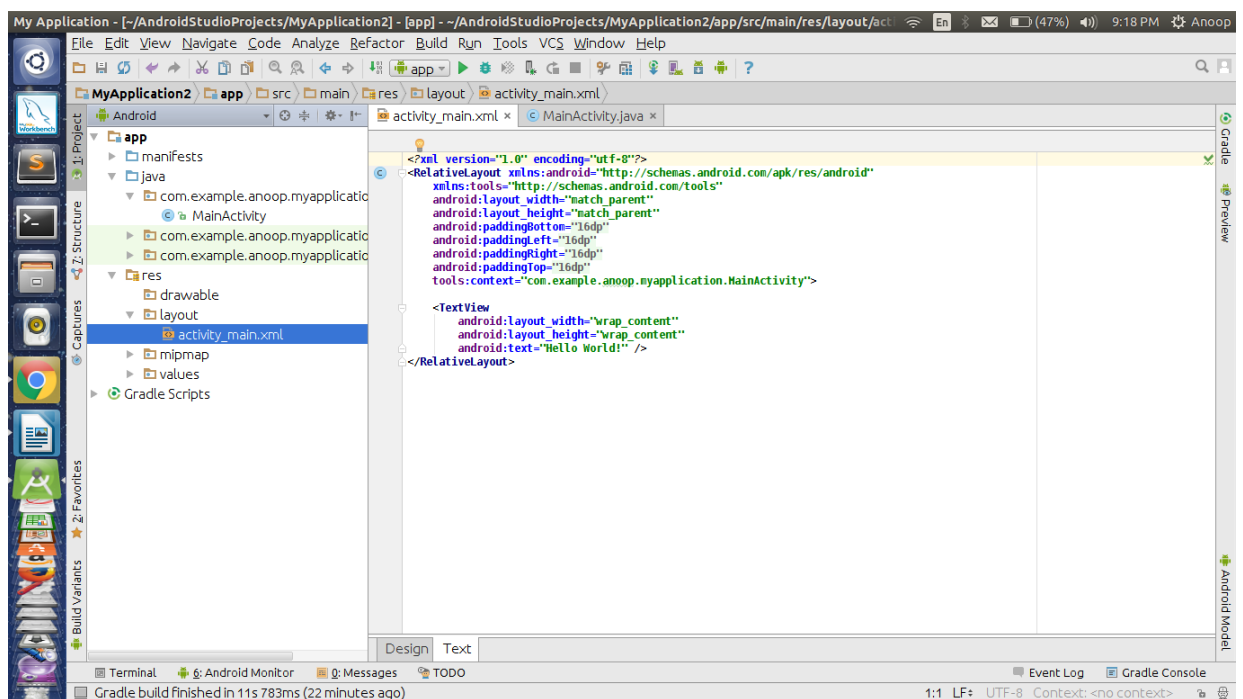
Android | Running your first Android app

- After successfully [Setting up an Android project](#), all of the default files are created with default code in them. Let us look at this default code and files and try to run the default app created.
 - The panel on the left side of the android studio window has all the files that the app includes. Under the **java folder**, observe the first folder containing the java file of your project.



- For every activity, a “.java” file and a “.xml” file is created. In this case for MainActivity, “MainActivity.java” and “activity_main.xml” are created. The above java file shows us the default code that is present when an app is created. An activity is created that extends AppCompatActivity class.

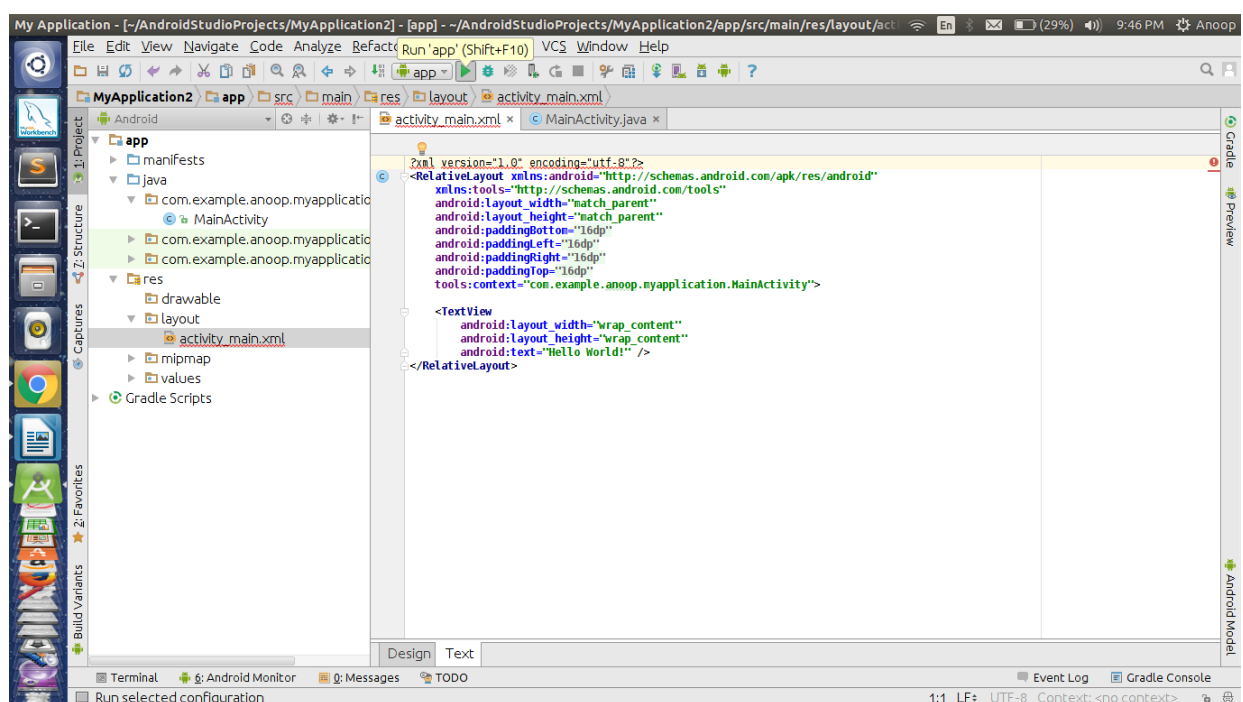
The “res” folder contains “layout” subfolder, which includes the *xml files* of the projects.



You can find the `activity_main.xml` file under the layout folder. This is the XML file corresponding to the MainActivity. There is an `onCreate` function that overrides a function of `AppCompatActivity` class.

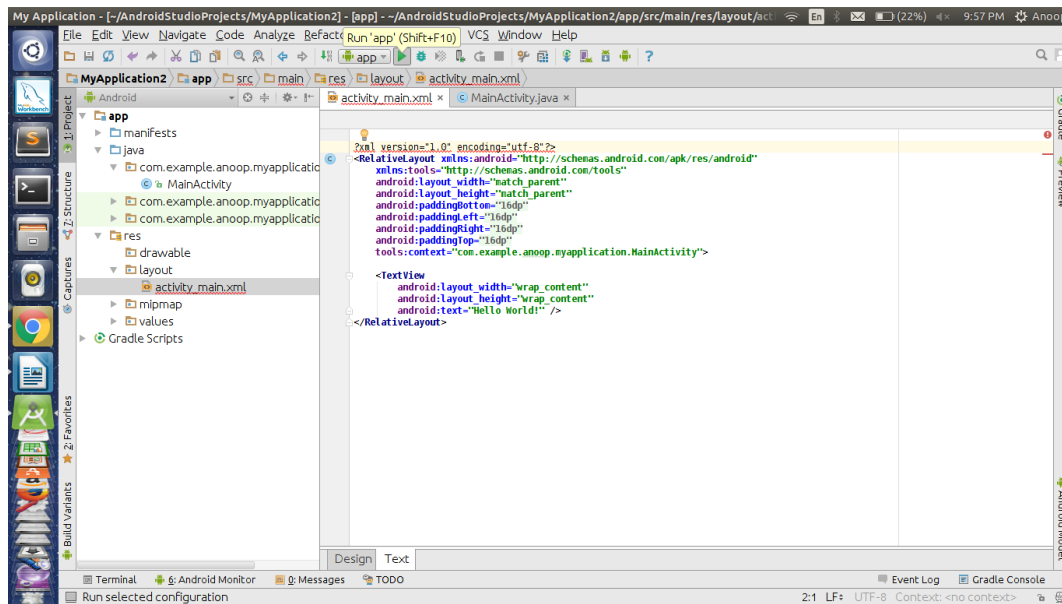
`onCreate(Bundle)` is where you initialize your activity. When the activity is first started, then both `onCreate()` methods are called. But after the first start of Activity, the `onCreate()` of application will not be called for subsequent runs.

- Now, consider the **activity_main.xml** file, it contains various tags similar to HTML. The first tag ensures the version. The second tag is usually the Layout tag. There are various types of Layouts but for now, let us go with the default *RelativeLayout*. This is a layout that places the widgets relative to screen size.



Now, click the “Run” option at the Toolbar at the top. You can observe the option being highlighted in the image below.

- You would get a pop-up as in the image below.

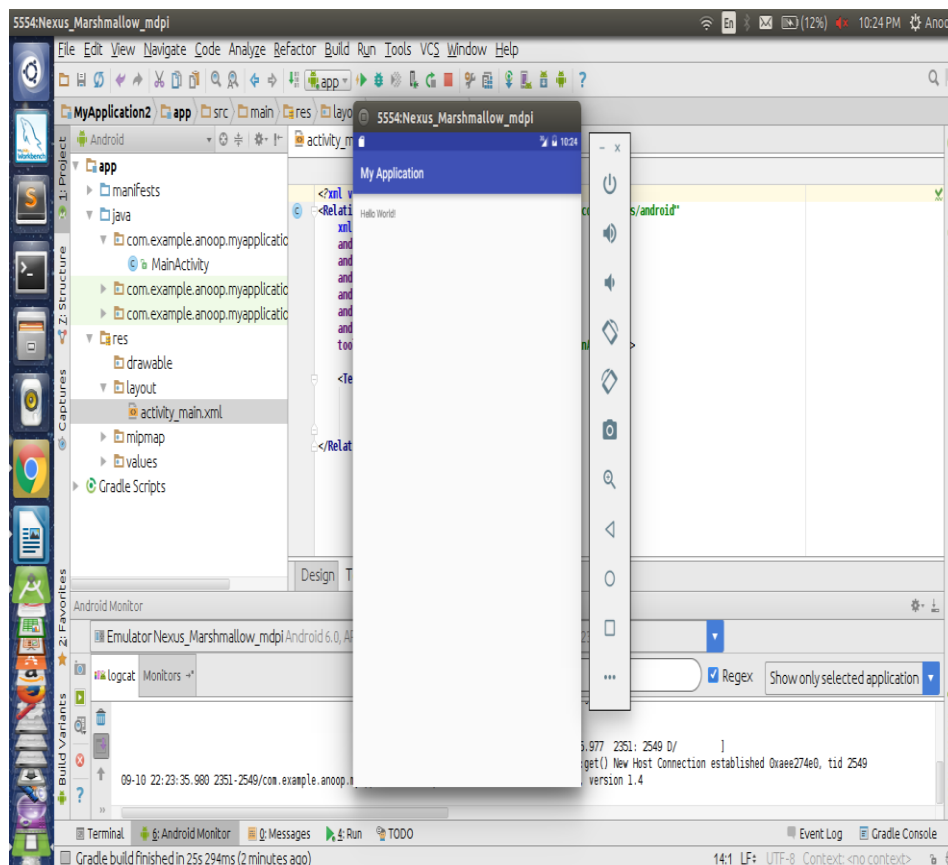


- You can either choose the emulator or you can connect your phone and find them listed under Connected Devices but for this you must enable the developer options in your phone and set the USB debugging mode on.

Once done, click on OK.

- Usually, the emulator consumes a lot of RAM. The more RAM size you have, the faster your emulator will work. Generally, 4GB is the descent RAM size. Size more than that would increase the performance of your emulator.

The image below shows the working of the first app, *My Application*. You can find all the basic functionalities that your phone has, on the emulator, like Home button, back button, power, etc.



Android Virtual Devices

The android emulator creates an Android Virtual Device (AVD) based on the Android Version and hardware specifications of the device we are stimulating. We must make an AVD for all the devices we plan to run the application on for effective app testing. Each AVD can be considered an independent device.

Running the Android Emulator Directly in Android Studio

The Android Studio runs the Android Emulator inside it by default. It has many advantages to it. It allows the user to conserve screen real estate, organize your IDE and emulator workflow in a single window and quickly move through the emulator and editor window.



Setting up A Physical Device

Firstly, we need to decide whether we should use Wi-Fi or USB to connect our device. After that, we need to follow these steps:

1. Go to the settings tab on the device and open it.
2. Select developer options and go to USB Debugging.
3. Set up your system and connect your device to it.

Connecting Through USB

If you decide to connect to your device using USB, follow these steps:

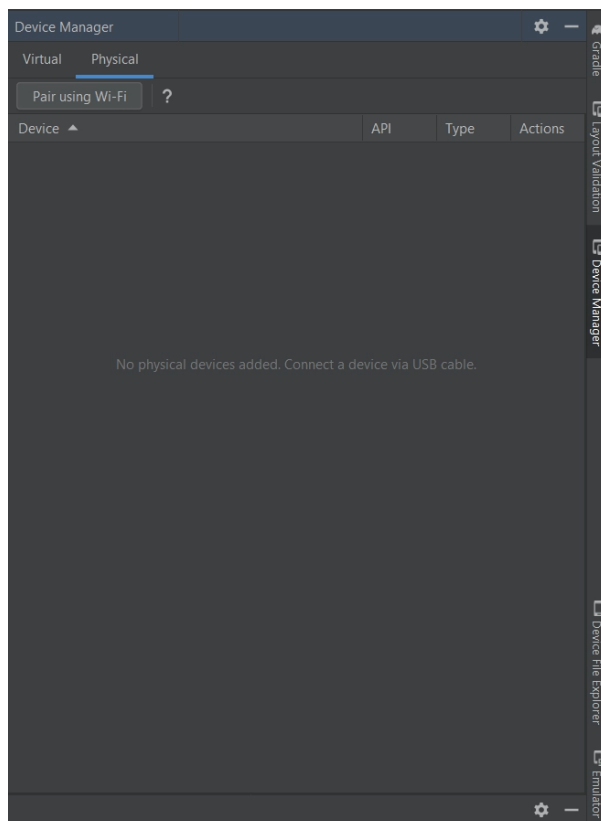
1. Set up your device and plug it over USB.
2. Now click Run in Android Studio. Now you can build and run your App on the device

Connecting Through Wi-fi

Using the Android debugging bridge, you can debug your App wirelessly on Android 11 and later versions. It aids us in deploying the App to multiple remote devices without having to connect the device physically.

To pair the device to your systems, follow the following steps:

1. Connect the workstation and the device to the same wireless network.
2. The device must be running on Android 11 or a higher version.
3. Download Android Studio Bumblebee Canary if you don't have it already.
4. Update the SDK Platform Tools to the latest version.
5. If all of the requirements are now up to date. Open Android Studio.
6. Go to the run configuration drop-down menu and select **Pair Devices using Wi-fi**.
7. Enable Developer's Option on your device.
8. Enable Debugging over Wifi on your device.
9. Select debugging and then pair your device. You can pair your device with a QR Code or by using Pairing Code.
10. When successfully paired, you can start by deploying your application and then work on it.



-----XXX-----

Device Manager

Virtual

Physical

Pair using Wi-Fi ?

Device ▲	API	Type	Actions
No physical devices added. Connect a device via USB cable.			

⚙️

—

Gradle

Layout Validation

Device Manager

Device File Explorer

Emulator